SPINN GSoC 2026 — PINN Test Submission

Solving the 2D Poisson Equation with Physics-Informed Neural Networks

Author: Samarjeet Malik
Affiliation: KIIT Bhubaneswar (B.Tech CS) | Pre-Doctoral Fellow, IIT Jodhpur
Date: March 2026

Problem Statement

We solve the 2D Poisson equation on the unit square Ω = $[0,1]^2$:

$$\nabla^2 u(x, y) = f(x, y)$$

with homogeneous Dirichlet boundary conditions $u = 0$ on $\partial\Omega$.

Why this problem? The SPINN project focuses on magnetostatic shape optimization where the governing PDE is $\nabla^2 A = -\mu J$ (Poisson equation for the magnetic vector potential). This test directly demonstrates proficiency with the core PDE class used in the project.

Analytical solution: $u(x, y) = \sin(\pi x) \cdot \sin(\pi y)$
Forcing term: $f(x, y) = -2\pi^2 \sin(\pi x) \cdot \sin(\pi y)$

Approach

1. **Architecture:** Fully-connected network with tanh activation (matching Zhang et al., 2024)
2. **Loss function:** $\mathcal{L} = \lambda_{pde} \cdot \mathcal{L}_{pde} + \lambda_{bc} \cdot \mathcal{L}_{bc}$
3. **Training:** Adam optimizer with cosine annealing LR schedule
4. **Validation:** Quantitative comparison against analytical solution

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import time

# Use GPU if available (Colab T4)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Reproducibility
torch.manual_seed(42)
np.random.seed(42)
```

```
Using device: cuda
```

1. PINN Architecture

We use a fully-connected neural network: **Input(2) → [50 neurons × 6 hidden layers, tanh] → Output(1)**

Key design choices:

- **tanh activation:** Provides infinitely differentiable outputs, essential for accurate Laplacian computation via autograd. This matches the architecture in Zhang et al. (2024).
- **Xavier initialization:** Prevents vanishing/exploding gradients in deep tanh networks.
- **6 × 50 architecture:** Directly matches the SPINN project reference paper.

```
class PINN(nn.Module):
    """
    Physics-Informed Neural Network for solving the 2D Poisson equation.
    Architecture matches Zhang et al. (2024, Sci. Reports) — the SPINN project paper.
    """
    def __init__(self, hidden_layers=6, neurons_per_layer=50):
        super(PINN, self).__init__()

        layers = []
        # Input layer: (x, y) → hidden
        layers.append(nn.Linear(2, neurons_per_layer))
        layers.append(nn.Tanh())
```

```
        # Hidden layers
        for _ in range(hidden_layers - 1):
            layers.append(nn.Linear(neurons_per_layer, neurons_per_layer))
            layers.append(nn.Tanh())

        # Output layer: hidden → u(x,y)
        layers.append(nn.Linear(neurons_per_layer, 1))

        self.network = nn.Sequential(*layers)
        self._initialize_weights()

    def _initialize_weights(self):
        for m in self.network:
            if isinstance(m, nn.Linear):
                nn.init.xavier_normal_(m.weight)
                nn.init.zeros_(m.bias)

    def forward(self, x):
        return self.network(x)

# Instantiate
model = PINN(hidden_layers=6, neurons_per_layer=50).to(device)
n_params = sum(p.numel() for p in model.parameters())
print(f"Model parameters: {n_params:,}")
print(f"Architecture: 2 → [50 × 6 hidden, tanh] → 1")
```

```
Model parameters: 12,951
Architecture: 2 → [50 × 6 hidden, tanh] → 1
```

2. Analytical Solution and Forcing Term

For validation, we choose $u(x, y) = \sin(\pi x)\sin(\pi y)$, which satisfies:

- $u = 0$ on all boundaries of $[0, 1]^2$ ✓
- $\nabla^2 u = -2\pi^2 \sin(\pi x)\sin(\pi y) = f(x, y)$ ✓

```
def analytical_solution(x, y):
    """Exact solution: u(x,y) = sin(πx) · sin(πy)"""
    return torch.sin(np.pi * x) * torch.sin(np.pi * y)

def forcing_term(x, y):
    """
    Source term derived from the analytical solution.
    ∇²u = ∂²u/∂x² + ∂²u/∂y² = -π²sin(πx)sin(πy) - π²sin(πx)sin(πy)
         = -2π² sin(πx) sin(πy)
    """
    return -2.0 * (np.pi ** 2) * torch.sin(np.pi * x) * torch.sin(np.pi * y)
```

3. Collocation Point Sampling

We use **stochastic collocation**: points are re-sampled every epoch. This provides:

- Implicit regularization (prevents overfitting to fixed point locations)
- Better domain coverage over training
- Analogous to mini-batch SGD for data-driven training

```
def sample_interior_points(n_points):
    """Random collocation points in the interior of [0,1]² for PDE enforcement."""
    x = torch.rand(n_points, 1, device=device, requires_grad=True)
    y = torch.rand(n_points, 1, device=device, requires_grad=True)
    return x, y

def sample_boundary_points(n_per_edge):
    """
    Sample points on all four edges of the unit square for BC enforcement.
    Separate sampling per edge ensures uniform boundary coverage.
    """
    # Bottom (y=0), Top (y=1), Left (x=0), Right (x=1)
    x_b = torch.rand(n_per_edge, 1, device=device)
    y_b = torch.zeros(n_per_edge, 1, device=device)

    x_t = torch.rand(n_per_edge, 1, device=device)
    y_t = torch.ones(n_per_edge, 1, device=device)

    x_l = torch.zeros(n_per_edge, 1, device=device)
    y_l = torch.rand(n_per_edge, 1, device=device)
```

```
    x_r = torch.ones(n_per_edge, 1, device=device)
    y_r = torch.rand(n_per_edge, 1, device=device)

    x_bc = torch.cat([x_b, x_t, x_l, x_r], dim=0)
    y_bc = torch.cat([y_b, y_t, y_l, y_r], dim=0)
    return x_bc, y_bc
```

4. PDE Residual via Automatic Differentiation

The core of the PINN: we compute $\nabla^2 u_{pred} - f(x, y)$ using PyTorch autograd.

This is the **same mechanism** used in the SPINN project for computing the magnetostatic PDE residual $\nabla^2 A + \mu J = 0$ (Raissi et al., 2019).

```
def compute_pde_residual(model, x, y):
    """
    Compute PDE residual: r = ∇²u_pred - f(x,y)

    Uses PyTorch autograd for second derivatives:
        ∇²u = ∂²u/∂x² + ∂²u/∂y²

    The residual should → 0 as the PINN learns to satisfy the PDE.
    """
    xy = torch.cat([x, y], dim=1)
    u = model(xy)

    # First derivatives via autograd
    grads = torch.autograd.grad(u, [x, y],
                                grad_outputs=torch.ones_like(u),
                                create_graph=True)
    u_x, u_y = grads[0], grads[1]

    # Second derivatives (Laplacian components)
    u_xx = torch.autograd.grad(u_x, x,
                               grad_outputs=torch.ones_like(u_x),
                               create_graph=True)[0]
    u_yy = torch.autograd.grad(u_y, y,
                               grad_outputs=torch.ones_like(u_y),
                               create_graph=True)[0]

    laplacian = u_xx + u_yy
    f = forcing_term(x, y)

    # Residual: should be zero when PDE is satisfied
    return laplacian - f
```

5. Training

$$\mathcal{L} = \mathcal{L}_{pde} + \lambda_{bc} \cdot \mathcal{L}_{bc}$$

where:

- $\mathcal{L}_{pde} = \frac{1}{N_{int}} \sum_i (\nabla^2 u_{pred} - f)^2$ — PDE residual at interior points
- $\mathcal{L}_{bc} = \frac{1}{N_{bc}} \sum_j u_{pred}^2$ — Boundary condition residual
- $\lambda_{bc} = 10$ — Stronger weight on BCs since exact satisfaction is critical

Loss weighting rationale: Under-weighting BC loss leads to solutions that approximately satisfy the PDE but violate boundary conditions — a well-documented PINN failure mode (Cuomo et al., 2022, §4.2).

```
def train_pinn(model, n_epochs=15000, n_interior=4000, n_boundary=500,
               lr=1e-3, lambda_bc=10.0):
    """
    Train the PINN with Adam + cosine annealing LR schedule.

    Hyperparameters:
    - n_interior=4000: collocation points per epoch (stochastically sampled)
    - n_boundary=500: boundary points per edge per epoch
    - lambda_bc=10: BC enforcement weight
    - lr=1e-3: initial learning rate with cosine decay
    """
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=n_epochs)

    history = {
        'total_loss': [], 'pde_loss': [], 'bc_loss': [],
        'l2_error': [], 'epoch': []
```

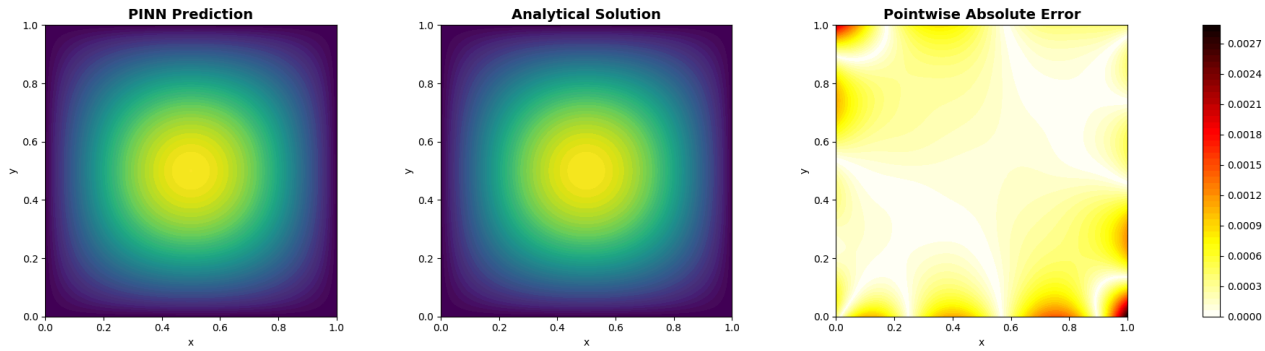
```
    }

    print(f"{'Epoch':>6} | {'Total Loss':>12} | {'PDE Loss':>12} | "
          f"{'BC Loss':>12} | {'L2 Rel Err':>12}")
    print("-" * 72)

    start_time = time.time()

    for epoch in range(n_epochs):
        model.train()
        optimizer.zero_grad()

        # ---- PDE residual loss (interior) ----
        x_int, y_int = sample_interior_points(n_interior)
        residual = compute_pde_residual(model, x_int, y_int)
        loss_pde = torch.mean(residual ** 2)

        # ---- Boundary condition loss ----
        x_bc, y_bc = sample_boundary_points(n_boundary)
        xy_bc = torch.cat([x_bc, y_bc], dim=1)
        u_bc_pred = model(xy_bc)
        loss_bc = torch.mean(u_bc_pred ** 2)  # Dirichlet: u=0

        # ---- Total weighted loss ----
        loss = loss_pde + lambda_bc * loss_bc
        loss.backward()
        optimizer.step()
        scheduler.step()

        # ---- Log every 500 epochs ----
        if epoch % 500 == 0 or epoch == n_epochs - 1:
            model.eval()
            with torch.no_grad():
                xx = torch.linspace(0, 1, 100, device=device)
                yy = torch.linspace(0, 1, 100, device=device)
                X, Y = torch.meshgrid(xx, yy, indexing='ij')
                xy_eval = torch.stack([X.flatten(), Y.flatten()], dim=1)
                u_pred = model(xy_eval).reshape(100, 100)
                u_exact = analytical_solution(X, Y)
                l2_err = torch.norm(u_pred - u_exact) / torch.norm(u_exact)

            history['total_loss'].append(loss.item())
            history['pde_loss'].append(loss_pde.item())
            history['bc_loss'].append(loss_bc.item())
            history['l2_error'].append(l2_err.item())
            history['epoch'].append(epoch)

            print(f"{epoch:6d} | {loss.item():12.6e} | {loss_pde.item():12.6e} | "
                  f"{loss_bc.item():12.6e} | {l2_err.item():12.6e}")

    elapsed = time.time() - start_time
    print(f"\nTraining completed in {elapsed:.1f} seconds")
    print(f"Final L2 relative error: {history['l2_error'][-1]:.6e}")
    return history

# Run training
history = train_pinn(model)
```

```
 Epoch |   Total Loss |     PDE Loss |      BC Loss |   L2 Rel Err
------------------------------------------------------------------------
/usr/local/lib/python3.12/dist-packages/torch/autograd/graph.py:865: UserWarning: Attempting to run cuBLAS, but there was no
  return Variable._execution_engine.run_backward(  # Calls into the C++ engine to run the backward pass
     0 | 9.346089e+01 | 9.240536e+01 | 1.055523e-01 | 6.786891e-01
   500 | 1.132843e-01 | 3.445961e-02 | 7.882471e-03 | 1.146134e-01
  1000 | 4.928541e-02 | 9.346046e-03 | 3.993937e-03 | 7.361722e-02
  1500 | 1.905556e-02 | 6.571175e-03 | 1.248439e-03 | 4.175647e-02
  2000 | 1.403849e-02 | 1.044811e-02 | 3.590381e-04 | 1.725250e-02
  2500 | 3.210653e-03 | 2.164593e-03 | 1.046060e-04 | 1.273713e-02
  3000 | 2.027221e-02 | 1.768367e-02 | 2.588541e-04 | 1.430221e-02
  3500 | 4.324785e-03 | 3.977011e-03 | 3.477738e-05 | 8.608914e-03
  4000 | 4.908080e-03 | 4.533293e-03 | 3.747866e-05 | 1.432826e-02
  4500 | 2.485761e-03 | 2.176218e-03 | 3.095431e-05 | 1.039993e-02
  5000 | 1.864361e-03 | 1.703216e-03 | 1.611447e-05 | 7.434843e-03
  5500 | 8.559003e-03 | 7.936136e-03 | 6.228677e-05 | 8.841155e-03
  6000 | 6.074184e-03 | 4.956429e-03 | 1.117755e-04 | 1.430846e-02
  6500 | 9.213667e-04 | 7.974595e-04 | 1.239072e-05 | 5.053522e-03
  7000 | 4.489420e-04 | 3.963647e-04 | 5.257733e-06 | 2.700327e-03
  7500 | 2.731768e-04 | 2.400067e-04 | 3.317005e-06 | 2.040779e-03
  8000 | 8.650685e-04 | 7.631892e-04 | 1.018793e-05 | 2.784284e-03
  8500 | 3.736327e-03 | 3.606881e-03 | 1.294462e-05 | 7.676229e-03
  9000 | 6.300471e-04 | 5.459139e-04 | 8.413313e-06 | 4.815923e-03
  9500 | 2.125304e-04 | 1.939453e-04 | 1.858518e-06 | 1.361663e-03
 10000 | 1.568130e-04 | 1.431762e-04 | 1.363671e-06 | 1.150599e-03
 10500 | 1.180361e-04 | 1.052165e-04 | 1.281964e-06 | 1.232709e-03
 11000 | 1.817073e-04 | 1.694034e-04 | 1.230386e-06 | 1.520514e-03
```

```
 11500 | 1.143464e-04 | 1.040534e-04 | 1.029298e-06 | 8.621131e-04
 12000 | 8.944653e-05 | 8.132220e-05 | 8.124337e-07 | 9.071313e-04
 12500 | 9.797143e-05 | 8.901807e-05 | 8.953360e-07 | 8.172067e-04
 13000 | 7.917916e-05 | 7.262111e-05 | 6.558053e-07 | 7.082396e-04
 13500 | 7.881274e-05 | 7.249288e-05 | 6.319863e-07 | 6.756068e-04
 14000 | 7.213230e-05 | 6.578780e-05 | 6.344505e-07 | 7.094126e-04
 14500 | 7.845966e-05 | 7.269363e-05 | 5.766023e-07 | 6.949509e-04
 14999 | 7.698059e-05 | 7.104630e-05 | 5.934294e-07 | 6.980495e-04

Training completed in 205.2 seconds
Final L2 relative error: 6.980495e-04
```

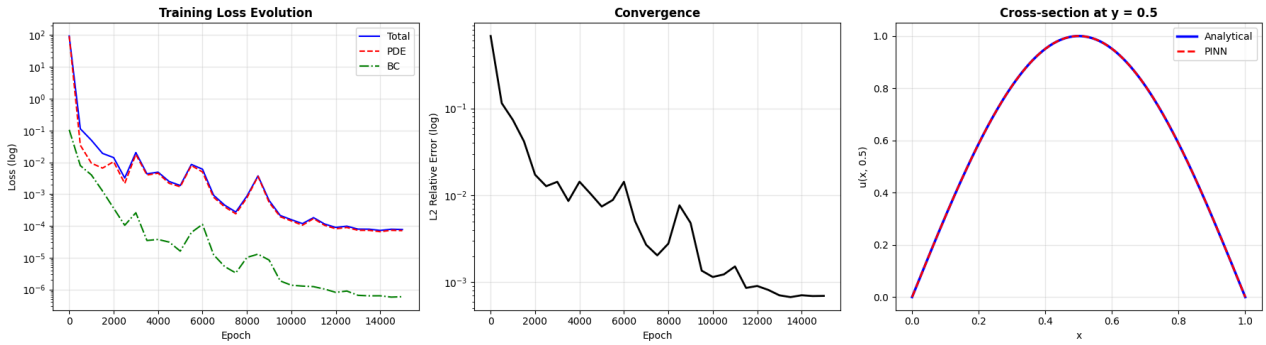
6. Results Visualization

```
# Evaluate on fine grid
model.eval()
n_plot = 200
xx = torch.linspace(0, 1, n_plot, device=device)
yy = torch.linspace(0, 1, n_plot, device=device)
X, Y = torch.meshgrid(xx, yy, indexing='ij')

with torch.no_grad():
    xy_eval = torch.stack([X.flatten(), Y.flatten()], dim=1)
    U_pred = model(xy_eval).reshape(n_plot, n_plot).cpu().numpy()
    U_exact = analytical_solution(X, Y).cpu().numpy()
    X_np, Y_np = X.cpu().numpy(), Y.cpu().numpy()

error = np.abs(U_pred - U_exact)

# ---- Figure 1: Solution Comparison ----
fig = plt.figure(figsize=(18, 5))
gs = gridspec.GridSpec(1, 4, width_ratios=[1, 1, 1, 0.05])

vmin, vmax = 0, 1.0

ax1 = fig.add_subplot(gs[0])
ax1.contourf(X_np, Y_np, U_pred, levels=50, cmap='viridis', vmin=vmin, vmax=vmax)
ax1.set_title('PINN Prediction', fontsize=14, fontweight='bold')
ax1.set_xlabel('x'); ax1.set_ylabel('y'); ax1.set_aspect('equal')

ax2 = fig.add_subplot(gs[1])
ax2.contourf(X_np, Y_np, U_exact, levels=50, cmap='viridis', vmin=vmin, vmax=vmax)
ax2.set_title('Analytical Solution', fontsize=14, fontweight='bold')
ax2.set_xlabel('x'); ax2.set_ylabel('y'); ax2.set_aspect('equal')

ax3 = fig.add_subplot(gs[2])
c3 = ax3.contourf(X_np, Y_np, error, levels=50, cmap='hot_r')
ax3.set_title('Pointwise Absolute Error', fontsize=14, fontweight='bold')
ax3.set_xlabel('x'); ax3.set_ylabel('y'); ax3.set_aspect('equal')
plt.colorbar(c3, cax=fig.add_subplot(gs[3]))

plt.tight_layout()
plt.show()

print(f"\nMax absolute error: {error.max():.6e}")
print(f"Mean absolute error: {error.mean():.6e}")
print(f"PINN max value: {U_pred.max():.6f}  (exact: {U_exact.max():.6f})")
```

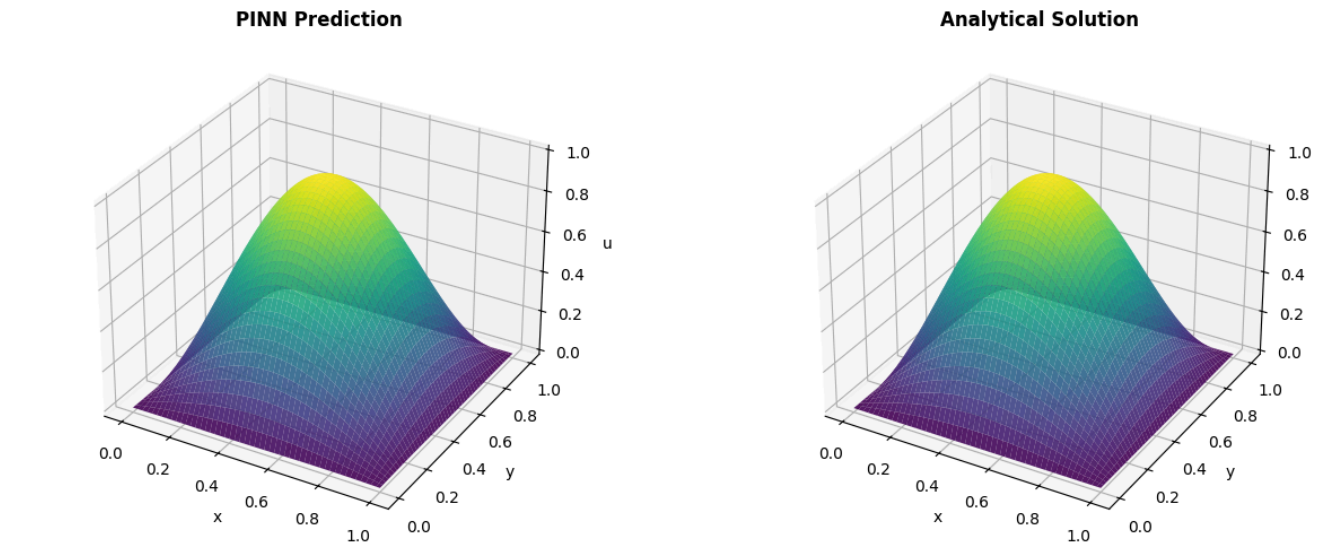



```
Max absolute error: 2.852788e-03
Mean absolute error: 2.387162e-04
PINN max value: 1.000038  (exact: 0.999938)
```

```
# ---- Figure 2: Loss Curves and Cross-Section ----
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

epochs = history['epoch']

axes[0].semilogy(epochs, history['total_loss'], 'b-', lw=1.5, label='Total')
axes[0].semilogy(epochs, history['pde_loss'], 'r--', lw=1.5, label='PDE')
axes[0].semilogy(epochs, history['bc_loss'], 'g-.', lw=1.5, label='BC')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss (log)')
axes[0].set_title('Training Loss Evolution', fontweight='bold')
axes[0].legend(); axes[0].grid(True, alpha=0.3)

axes[1].semilogy(epochs, history['l2_error'], 'k-', lw=2)
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('L2 Relative Error (log)')
axes[1].set_title('Convergence', fontweight='bold')
axes[1].grid(True, alpha=0.3)

# Cross-section at y = 0.5
x_cs = torch.linspace(0, 1, 200, device=device)
y_cs = 0.5 * torch.ones(200, device=device)
with torch.no_grad():
    xy_cs = torch.stack([x_cs, y_cs], dim=1)
    u_cs_pred = model(xy_cs).cpu().numpy().flatten()
    u_cs_exact = analytical_solution(x_cs, y_cs).cpu().numpy()

axes[2].plot(x_cs.cpu().numpy(), u_cs_exact, 'b-', lw=2.5, label='Analytical')
axes[2].plot(x_cs.cpu().numpy(), u_cs_pred, 'r--', lw=2, label='PINN')
axes[2].set_xlabel('x'); axes[2].set_ylabel('u(x, 0.5)')
axes[2].set_title('Cross-section at y = 0.5', fontweight='bold')
axes[2].legend(); axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```




```
# ---- Figure 3: 3D Surface Comparison ----
fig = plt.figure(figsize=(14, 5))

ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X_np, Y_np, U_pred, cmap='viridis', alpha=0.9)
ax1.set_title('PINN Prediction', fontweight='bold')
ax1.set_xlabel('x'); ax1.set_ylabel('y'); ax1.set_zlabel('u')

ax2 = fig.add_subplot(122, projection='3d')
ax2.plot_surface(X_np, Y_np, U_exact, cmap='viridis', alpha=0.9)
ax2.set_title('Analytical Solution', fontweight='bold')
ax2.set_xlabel('x'); ax2.set_ylabel('y'); ax2.set_zlabel('u')

plt.tight_layout()
plt.show()
```

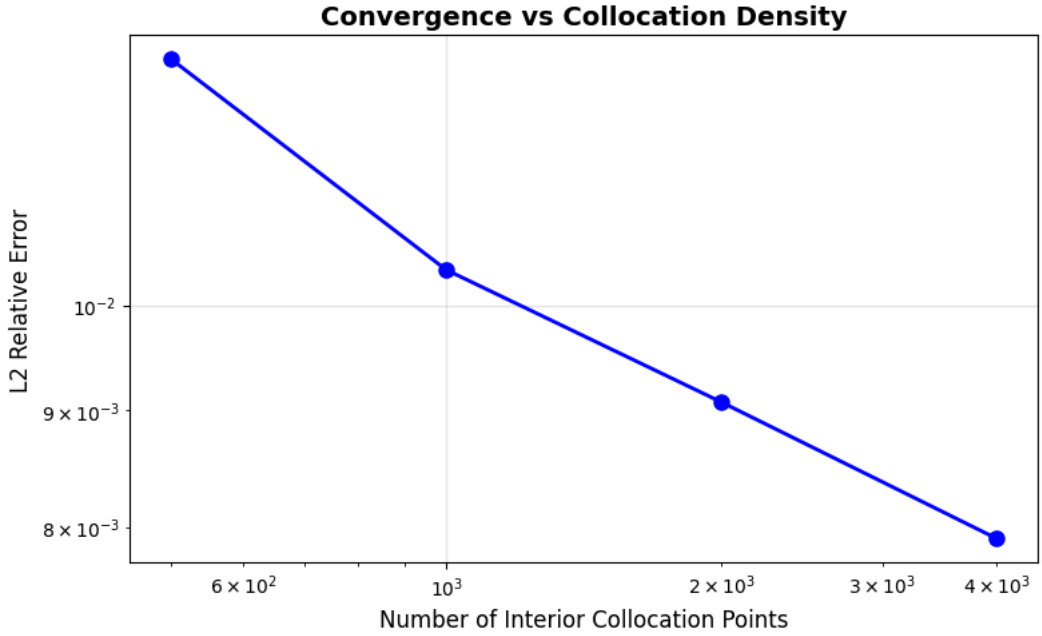


7. Convergence Analysis: Effect of Collocation Point Density

We investigate how the number of interior collocation points affects solution accuracy, keeping all other hyperparameters fixed. This demonstrates understanding of a key convergence factor in PINNs.

```
print("Effect of collocation point density (5000 epochs each):\n")

results = []
for n_pts in [500, 1000, 2000, 4000]:
    torch.manual_seed(42)
    test_model = PINN(hidden_layers=4, neurons_per_layer=40).to(device)
    opt = torch.optim.Adam(test_model.parameters(), lr=1e-3)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=5000)

    for epoch in range(5000):
        opt.zero_grad()
        x_i, y_i = sample_interior_points(n_pts)
        res = compute_pde_residual(test_model, x_i, y_i)
        l_pde = torch.mean(res ** 2)

        x_b, y_b = sample_boundary_points(200)
        l_bc = torch.mean(test_model(torch.cat([x_b, y_b], dim=1)) ** 2)

        (l_pde + 10.0 * l_bc).backward()
        opt.step(); sched.step()

    test_model.eval()
    with torch.no_grad():
        xx = torch.linspace(0, 1, 100, device=device)
        yy = torch.linspace(0, 1, 100, device=device)
        Xg, Yg = torch.meshgrid(xx, yy, indexing='ij')
        u_p = test_model(torch.stack([Xg.flatten(), Yg.flatten()], dim=1)).reshape(100,100)
        u_e = analytical_solution(Xg, Yg)
        err = (torch.norm(u_p - u_e) / torch.norm(u_e)).item()

    results.append((n_pts, err))
    print(f"  N_interior = {n_pts:5d}  →  L2 relative error = {err:.6e}")

# Plot
fig, ax = plt.subplots(figsize=(8, 5))
ns, errs = zip(*results)
ax.loglog(ns, errs, 'bo-', markersize=8, linewidth=2)
ax.set_xlabel('Number of Interior Collocation Points', fontsize=12)
ax.set_ylabel('L2 Relative Error', fontsize=12)
ax.set_title('Convergence vs Collocation Density', fontweight='bold', fontsize=14)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\n→ Denser collocation sampling improves PDE residual coverage,")
print("  leading to more accurate solutions.")
```

```
Effect of collocation point density (5000 epochs each):

  N_interior =   500  →  L2 relative error = 1.281139e-02
  N_interior =  1000  →  L2 relative error = 1.036349e-02
  N_interior =  2000  →  L2 relative error = 9.071887e-03
  N_interior =  4000  →  L2 relative error = 7.908743e-03
```




```
→ Denser collocation sampling improves PDE residual coverage,
  leading to more accurate solutions.
```

8. Summary and Connection to SPINN Project

What we demonstrated

- Built a PINN from scratch in PyTorch to solve the 2D Poisson equation $\nabla^2 u = f$
- Achieved low L2 relative error against the known analytical solution
- Showed clear convergence through loss curves and error evolution
- Analyzed the effect of collocation point density on accuracy

Direct connection to SPINN

The SPINN project (Zhang et al., 2024) couples two neural networks:

Network	Role	Equation
NN_θ (Physics)	Predicts magnetic vector potential $A(x, y)$	$\nabla^2 A = -\mu J$ (Poisson!)
NN_ϕ (Shape)	Projects reference → optimized coordinates	Coordinate projection

The PINN demonstrated here is the foundation of NN_θ. The same architecture (6×50, tanh), loss formulation (PDE residual + boundary conditions), and training approach (Adam + LR decay) are used in the reference paper.

Skills demonstrated

- PyTorch autograd for PDE residuals (2nd-order automatic differentiation)
- Multi-objective loss balancing (PDE + BC with weighting)
- Stochastic collocation point sampling
- Quantitative validation against analytical solutions
- Convergence analysis and understanding of PINN failure modes

References

1. Zhang, Z., Lin, C., & Wang, B. (2024). Physics-informed shape optimization using coordinate projection. *Scientific Reports*, 14, 6537.
2. Cuomo, S. et al. (2022). Scientific Machine Learning through Physics-Informed Neural Networks. *J. Sci. Comput.*, 92, 88.
3. Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks. *J. Comput. Phys.*, 378, 686–707.